IN-MEMORY KEY-VALUE STORE (Redis-lite)

Phase 2 + Phase 3 — Command Parsing & Storage. C++20, POSIX Sockets | Files, relations, line-by-line, data structures.

1 — PROJECT STRUCTURE NOW (after modular refactor)

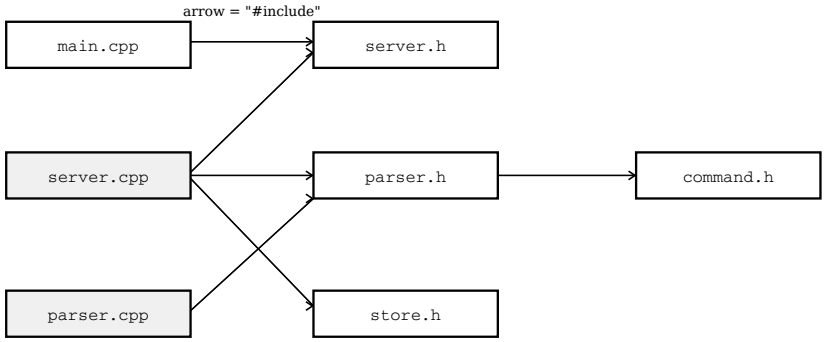
```
kvstore/
├── CMakeLists.txt      build config
├── include/            headers (.h) — "what"
│   ├── command.h       Command struct + enum
│   ├── parser.h        parse() declaration
│   ├── store.h         KVStore class declaration
│   └── server.h        Server class declaration
└── src/                sources (.cpp) — "how"
    ├── main.cpp        entry point (6 lines)
    ├── server.cpp      socket + loop + dispatch
    ├── parser.cpp      tokenize + validate
    └── store.cpp       set / get / del
```

What each file does

- **main.cpp** — makes a Server, runs it
- **server.h/.cpp** — TCP socket, accept, recv/send loop, command dispatch
- **command.h** — the parsed-command data shape (struct + enum)
- **parser.h/.cpp** — turn a raw string into a Command
- **store.h/.cpp** — the actual key→value storage (unordered_map)

.H VS .CPP .h = MENU (what exists). .cpp = KITCHEN (how it's made).

2 — FILE RELATIONS (who includes whom)



File	includes	why
main.cpp	server.h	needs the Server class
server.cpp	server.h, parser.h, store.h	runs socket, parses input, stores data
parser.cpp / parser.h	command.h	returns a Command struct
store.cpp	store.h	implements the KVStore class

3 — PHASE 2 : COMMAND PARSING

Goal: turn a raw string the client sent into a structured command the server can act on. (Understanding only — no storage yet.)

3.1 — command.h : the data shape

```
enum class CommandType { SET, GET, DEL, UNKNOWN };

struct Command {
    CommandType type  = CommandType::UNKNOWN;
    std::string key;
    std::string value;
    bool        valid = false;
    std::string error;
};
```

Part	Meaning
`enum class CommandType`	fixed list of allowed commands — type-safe (cannot be a random value)
`type = UNKNOWN`	which command; default UNKNOWN (safe)
`key / value`	the parsed key and value (value only for SET)
`valid = false`	did parsing succeed? default false (safe)
`error`	message when parsing fails

ORDER SLIP The customer speaks; the receptionist writes a slip: item, variant, size. The slip = the Command struct.

"SET name shivam" →

```
type  : SET
key   : "name"
value : "shivam"
valid : true
error : ""
```

"SET name" (bad) →

```
type  : UNKNOWN
key   : ""
value : ""
valid : false
error : "SET needs key and value"
```

3.2 — parser.h : the declaration

```
Command parse(const std::string& input);
```

Part	Meaning
`Command`	function returns a Command struct
`const std::string& input`	takes the raw text by reference (no copy) and promises not to change it
`; (no body)`	declaration only — real code lives in parser.cpp

CONST T& Pass the book itself (fast, no photocopy) with a promise "I won't write on it." Standard C++ for read-only inputs.

3.3 — parser.cpp : the logic (line by line)

```
Command parse(const std::string& input) {
    Command cmd;
    std::stringstream ss(input);
    std::string token;
    std::vector<std::string> tokens;
    while (ss >> token) tokens.push_back(token);

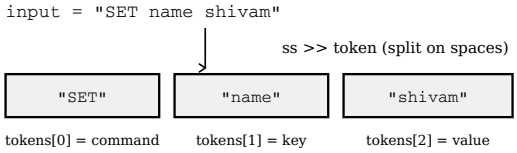
    if (tokens.empty()) { cmd.error = "empty command"; return cmd; }
    const std::string& cmdName = tokens[0];

    if (cmdName == "SET") {
        if (tokens.size() != 3) { cmd.error="SET needs key and value"; return cmd; }
        cmd.type=CommandType::SET; cmd.key=tokens[1]; cmd.value=tokens[2];
        cmd.valid=true; return cmd;
    }
    if (cmdName == "GET") { /* needs size 2 */ ... }
    if (cmdName == "DEL") { /* needs size 2 */ ... }
    cmd.error = "unknown command: " + cmdName;
    return cmd;
}
```

Line	Meaning
`stringstream ss(input)`	treat the string like an input stream; splits on whitespace
`while (ss >> token)`	pull one word at a time until empty
`tokens.push_back(token)`	store each word in a vector (dynamic array)
`tokens.empty()`	nothing typed → error, return

`cmdName = tokens[0]`	first word = the command name
`size != 3 / != 2`	validate token count per command
`cmd.valid = true`	mark success only after validation passes

tokenize visualization



3.4 — server.cpp : the parse loop (recv → parse → respond)

```
int bytes = recv(client_fd, buffer, sizeof(buffer), 0);
if (bytes <= 0) break;
std::string input(buffer, bytes);                 // bytes only, not 1024
while (!input.empty() && (input.back()=='\n'||input.back()=='\r'))
    input.pop_back();                              // trim newline from nc
Command cmd = parse(input);
// build response from cmd, then:
send(client_fd, response.c_str(), response.size(), 0);
```

Line	Meaning
`string input(buffer, bytes)`	build a string from the first *bytes* chars only (the rest of the 1024 are zero)
`trim \n / \r`	nc appends Enter; remove it or the last token becomes "shivam\n"
`parse(input)`	call the parser → Command struct back
`response.c_str(), .size()`	send() needs a char* + length; convert the string

4 — PHASE 3 : STORAGE (the real Redis behaviour)

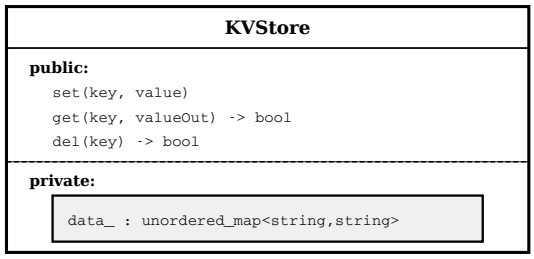
Goal: actually store/fetch/delete data in an `unordered_map`, wrapped in a KVStore class.

4.1 — store.h : the KVStore class

```
class KVStore {
public:
    void set(const std::string& key, const std::string& value);
    bool get(const std::string& key, std::string& valueOut);
    bool del(const std::string& key);
private:
    std::unordered_map<std::string,std::string> data_;
};
```

Member	Meaning
`set(key, value)`	store or update; returns nothing
`get(key, valueOut)`	returns bool (found?) and fills **valueOut** with the value
`del(key)`	returns bool (was something deleted?)
`private data_`	the actual map — hidden; only reachable through set/get/del (encapsulation)

GET() OUT-PARAMETER A function returns ONE thing, but get must answer two: "found?" and "the value." So: return the bool, and write the value into `valueOut` (a reference = the caller's own variable). You hand the receptionist YOUR box (&); they fill it and say "yes."



4.2 — store.cpp : set / get / del (line by line)

```
void KVStore::set(const std::string& key, const std::string& value) {
    data_[key] = value;                       // insert OR overwrite
}
bool KVStore::get(const std::string& key, std::string& valueOut) {
    auto it = data_.find(key);                // safe lookup
    if (it == data_.end()) return false;      // not found
    valueOut = it->second;                     // write value to caller's var
    return true;
}
bool KVStore::del(const std::string& key) {
    return data_.erase(key) > 0;              // erase returns count removed
}
```

Line	Meaning
`data_[key] = value`	if key absent → create; if present → overwrite
`data_.find(key)`	look up WITHOUT creating a blank entry (unlike `data_[key]`)
`it == data_.end()`	end() is the "not found" marker
`it->second`	iterator points to a pair: first = key, second = value
`valueOut = it->second`	fill the caller's variable (the out-parameter)
`erase(key) > 0`	erase returns how many removed: 1 = deleted, 0 = wasn't there

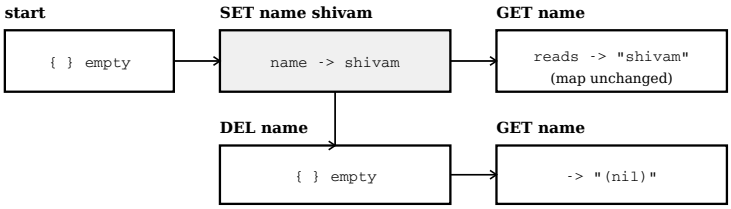
FIND VS [ ] `data_[key]` on a missing key silently creates a blank entry — a bug for GET. `find()` only looks; it never creates.

4.3 — server.cpp : dispatch (the switch that uses KVStore)

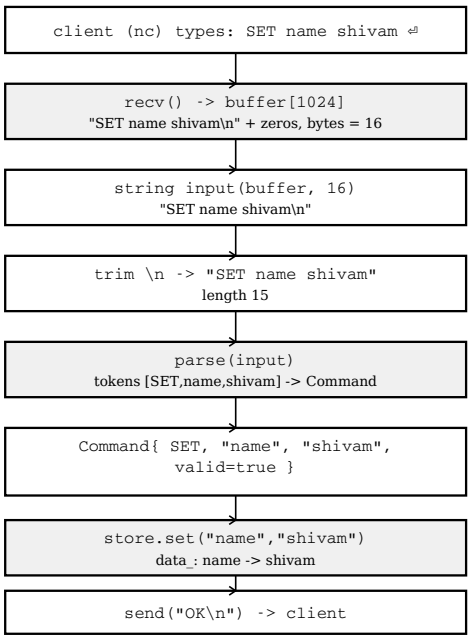
```
KVStore store;                       // ONE store, created before the loop
switch (cmd.type) {
  case SET: store.set(cmd.key, cmd.value); response = "OK\n"; break;
  case GET: { std::string v;
              response = store.get(cmd.key, v) ? v + "\n" : "(nil)\n"; break; }
  case DEL: response = store.del(cmd.key) ? "OK\n" : "(nil)\n"; break;
}
```

`KVStore store;` is declared OUTSIDE the loop so data survives between commands. Here the **valueOut** finally gets used: `store.get(cmd.key, v)` fills `v`, then `response = v`.

data_ state through SET / GET / DEL

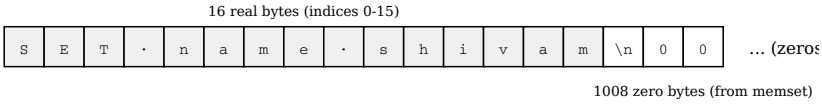

start
{ } empty
SET name shivam
name -> shivam
GET name
reads -> "shivam"
(map unchanged)
DEL name
{ } empty
GET name
-> "(nil)"

5 — END-TO-END EXECUTION FLOW : "SET name shivam"



6 — DATA STRUCTURE VISUALIZATIONS

6.1 — buffer[1024] right after recv (16 real bytes)



`sizeof(buffer)` = 1024 (size in bytes). `bytes` = 16 (what recv actually received). We build the string from **bytes**, not 1024.

6.2 — Command struct (filled)

```
Command
+-----------------------------+
| type   : CommandType::SET   |
| key    : "name"             |
| value  : "shivam"           |
| valid  : true               |
| error  : ""                 |
+-----------------------------+
```

unordered_map data_

```
+---------+-----------+
|  KEY    |  VALUE    |
+---------+-----------+
| "name"  | "shivam"  |
| "city"  | "surat"   |
+---------+-----------+
 find(key) -> iterator
   it->first  = key
   it->second = value
 end() = "not found"
```

7 — STATUS / ROADMAP

Done

- Phase 1 — TCP server (echo)
- Phase 2 — Command parsing (SET/GET/DEL + errors)
- Phase 3 — Storage (real SET/GET/DEL via unordered_map)
- Modular refactor: include/ + src/, CMake

Next (the "systems" depth)

- **Phase 4 — TTL** (background expiry thread)
- **Phase 5 — LRU** eviction (O(1) map + list)
- **Phase 6 — Concurrency**: many clients, sharded locking, P50/P99 + 50K ops/sec benchmark

Phases 1-3 are the foundation. Phases 4-6 are what stand out.

build & run

```
g++ -std=c++20 -Iinclude src/main.cpp src/server.cpp src/parser.cpp src/store.cpp -o server
./server                    # terminal 1
nc localhost 6379           # terminal 2
  SET name shivam  -> OK
  GET name         -> shivam
  GET city         -> (nil)
  DEL name         -> OK
  GET name         -> (nil)
```